

CSE 5095: Types and Programming Languages

Lambda Calculus Encodings

Stefan Muller

Thursday, Sept. 24

Note: This set of notes just gives the definitions we ended up with in class, with short reminders of the explanations. We gave more thorough explanations in class. For additional more thorough explanations, see *A Tutorial Introduction to the Lambda Calculus* by Raul Rojas, posted on HuskyCT (note that some of the definitions there are slightly different from those here, but the basic ideas are similar).

1 Numbers (Church Numerals)

Idea: represent a number n as a function that takes arguments s and z and applies s n times to z

$$\begin{aligned} 0 &\triangleq \lambda s. \lambda z. z \\ 1 &\triangleq \lambda s. \lambda z. s \ z \\ 2 &\triangleq \lambda s. \lambda z. s \ (s \ z) \\ &\dots \end{aligned}$$

1.1 Successor Function

Idea: Add one more application of the s argument.

$$S \triangleq \lambda n. \lambda s. \lambda z. s \ (n \ s \ z)$$

1.2 Addition

Idea: Apply S x times to y .

$$\text{add} \triangleq \lambda x. \lambda y. x \ S \ y$$

1.3 Multiplication

Idea: Apply add x times to 0.

$$\text{times} \triangleq \lambda x. \lambda y. x \ (\text{add} \ y) \ 0$$

2 (Church) Booleans

Idea: Represent `if e1 then e2 else e3` as $e1 \ e2 \ e3$; Booleans just select one of the two options.

$$\begin{aligned} \text{if } M_1 \ M_2 \ M_3 &\triangleq M_1 \ M_2 \ M_3 \\ \text{true} &\triangleq \lambda x. \lambda y. x \\ \text{false} &\triangleq \lambda x. \lambda y. y \end{aligned}$$

2.1 Zero test

Idea: Return $\lambda z.F$ applied n times to **true**.

$$\text{iszero} \triangleq \lambda n.n (\lambda z.\text{false}) \text{ true}$$

$\text{ifz } n \ t \ f$ is like **if** $n = 0$ **then** t **else** f

$$\text{ifz} \triangleq \lambda n.\lambda t.\lambda f.\text{if } (\text{iszero } n) \ t \ f$$

3 Pairs

Idea: We need a way of packaging two things together and saying which we want. We'll reuse **true** and **false** as the way of selecting one of two items.

$$\begin{aligned} (M_1, M_2) &\triangleq \lambda z.z \ M_1 \ M_2 \\ \text{fst} &\triangleq \lambda p.p \ \text{true} \\ \text{snd} &\triangleq \lambda p.p \ \text{false} \end{aligned}$$

4 Predecessor

Idea: Start with a function that returns $(n, n - 1)$ for n . This function counts up from 0, “remembering” the previous number in the second component of the pair.

$$\begin{aligned} \text{predpair} &\triangleq \lambda n.n (\lambda p.(\text{S } (\text{fst } p), \text{fst } p)) (0, 0) \\ \text{pred} &\triangleq \lambda n.\text{snd } (\text{predpair } n) \end{aligned}$$

5 Recursion

Idea: A “fixed point combinator”. We have $Y \ f \equiv f \ (Y \ f)$.

$$Y \triangleq \lambda f.(\lambda x.f \ (x \ x)) \ (\lambda x.f \ (x \ x))$$

6 Putting it all together: Factorial

Idea: Factorial takes an extra argument f which is the Factorial function used for the recursive call. We pass this whole thing to Y so that the Factorial function itself is passed as this argument.

$$\text{fact} \triangleq Y \ (\lambda f.\lambda n.\text{ifz } n \ 1 \ (\text{times } n \ (f \ (\text{pred } n))))$$